ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

DESARROLLO DE SOFTWARE SEGURO

Unidad 1: Introducción y Estudio de Vulnerabilidades

Tema 2: Amenazas a la seguridad del software

Profesor: Geovanny Cudco/agcudco@espe.edu.ec



CONTENIDOS

01 | Introducción

02 | 1.2. Amenazas a la seguridad del software

03 | 1.2.1. Amenazas a nivel de hardware

04 | 1.2.2. Amenazas a nivel de código

05 | 1.2.3. Lenguajes de script incrustados

06 | 1.2.4. Amenazas a nivel de diseño

07 | 1.2.5. Amenazas a nivel de arquitectura

1.2. Amenazas a la seguridad del software

Concepto y rasgos clave

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA



¿Qué es una amenaza?

Acción o circunstancia que aprovecha una vulnerabilidad para comprometer, atacar o invadir un sistema informático.

Rasgos principales



Puede originarse en fuentes **externas o internas.**



Explota **vulnerabilidades o fallos** del sistema.



Busca afectar la **confidencialidad, integridad o disponibilidad.**



Suele perseguir **daño, interrupción o beneficio ilícito.**

Fuentes: NIST (s. f.); OWASP Foundation (2025).

1.2. Amenazas a la seguridad del software

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA
ECUADOR

Tipos de amenazas

Las amenazas. pueden clasificarse, según su origen, en **amenazas externas e internas**.

Externas

- Provienen fuera de la organización.
- Ejemplos: malware, phishing, gusanos y troyanos.

Internas

- Se originan dentro de la organización.
- Ejemplos: acceso no autorizado, robo o fuga de datos.

Ambas pueden comprometer la operación, la información y la continuidad del servicio.

Fuentes: NIST (s. f.); OWASP Foundation (2025).

1.2. Amenazas a la seguridad del software

ESPE
UNVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Impactos y medidas de prevención

Impactos

- Pérdida o exposición de datos.
- Interrupción de servicios y tiempo de inactividad.
- Afectación económica y reputacional.

Prevención

- Mantener software y equipos actualizados.
- Aplicar validaciones y controles de acceso.
- Usar contraseñas seguras y proteger datos sensibles.
- Evitar abrir archivos o enlaces de remitentes desconocidos.

La seguridad del software requiere **prevención continua, actualización y buenas prácticas.**

Fuentes: OWASP Foundation (2025); Verizon (2026).

1.2. Amenazas a la seguridad del software

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

AMENNZAS

- Amenazas a nivel de Hardware
- Amenazas a nivel de Código
- Lenguajes de script incrustados
- Amenazas a nivel de Diseño
- Amenazas a nivel de Arquitectura



1.2.1. Amenazas a nivel de Hardware

Amenazas modernas y cadena de suministro

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Las amenazas modernas ya no se limitan al equipo final: pueden originarse en la fabricación, el transporte, los proveedores o dispositivos auxiliares.



Manipulación en tránsito

La modificación de dispositivos durante la cadena de suministro es un riesgo real.

Componentes o firmware comprometidos

Piezas falsificadas o firmware alterado introducen puertas traseras.

BMC e IoT/edge

Controladores auxiliares y equipos conectados aumentan la superficie de exposición.

Root of trust débil

Sin una base confiable, la verificación del arranque y del firmware se debilita.

Las dependencias con terceros exigen trazabilidad, validación de proveedores y verificación de integridad.

Fuentes: ENISA. (2025). ENISA Threat Landscape 2025; CISA, (2025). Threat Evaluation Working Group: Threat Scenarios v2.0.

1.2.1. Amenazas a nivel de Hardware

Panorama general

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

¿Qué son?

Son riesgos que afectan componentes físicos y firmware del equipo, como BIOS/UEFI, memoria, buses, puertos, almacenamiento y controladores embebidos.

Ya no se limitan al daño físico: incluyen firmware malicioso, manipulación de puertos y fallos de diseño.

Pueden ser **difíciles de detectar** porque operan por debajo del sistema operativo.

Comprometen confidencialidad, integridad, disponibilidad y el proceso de arranque.

NIST identifica **98 escenarios** de fallos de seguridad de hardware.



La seguridad del hardware requiere protección, detección y recuperación.

Fuentes: Mell, P., & Bojanova, I. (2024). NIST IR 8517; Rekhi, S., Amberiadis, K., Akib, A. A., & Srivastava, A. (2025). NIST CSWP 45.

1.2.1. Amenazas a nivel de Hardware

Superficies de ataque principales

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA
ECUADOR

SUPERFICIES

Firmware y arranque
BIOS/UEFI, Secure Boot, Option ROMs y bootloaders.

Memoria
RAM, DMA y ataques de modificación o lectura no autorizada.

Almacenamiento
HDD/SSD, robo físico, manipulación y firmware del disco.

Puertos e interfaces
USB, Thunderbolt, PCIe y puertos de depuración.

Controladores embebidos
BMC, TPM, sensores y microcontroladores auxiliares.



El hardware expone múltiples superficies de ataque, incluso antes de cargar el sistema operativo.

Fuentes: Mell, P., & Bojanova, I. (2024). NIST IR 8517; Regenscheid, A. (2018). NIST SP 800-193.

1.2.1. Amenazas a nivel de Hardware

Memoria, buses e interfaces físicas

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA
ECUADOR

RAM y DMA

El acceso directo a memoria puede permitir lectura o manipulación sin pasar por el sistema operativo.

Side-channel y Rowhammer

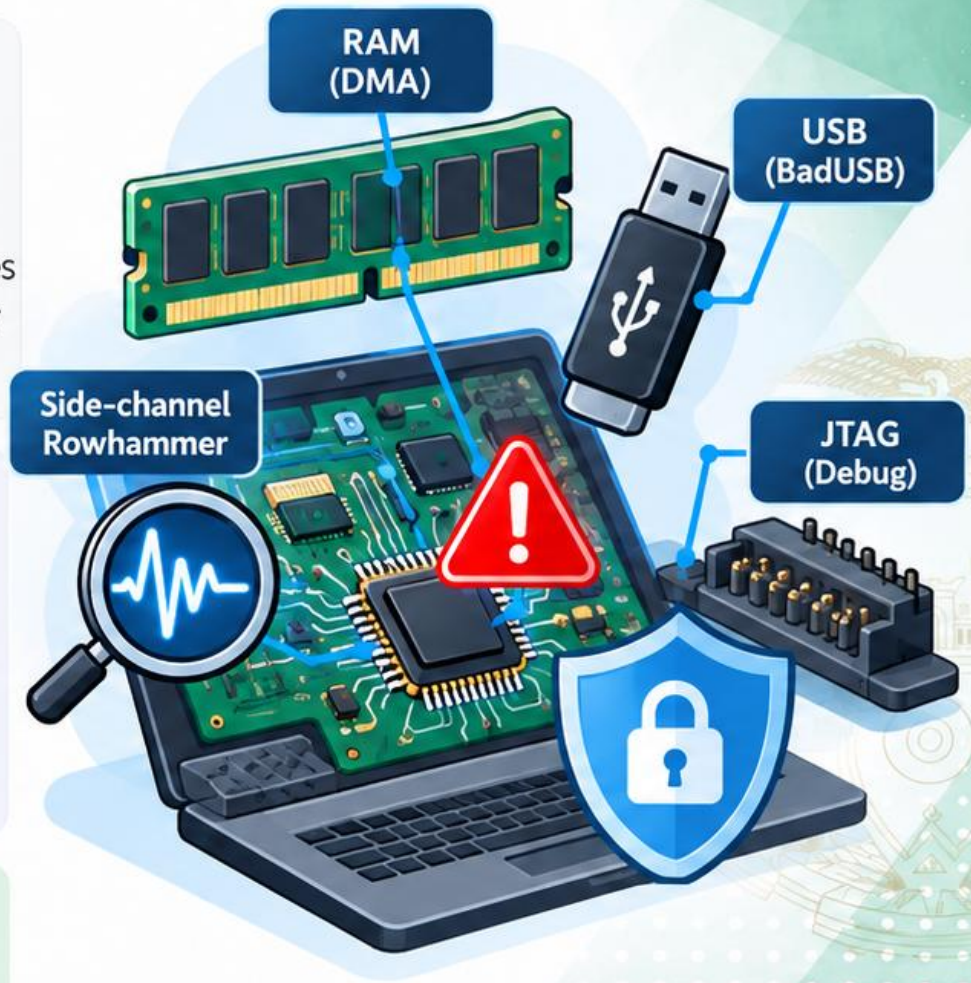
Algunas fallas explotan fugas físicas o perturbaciones eléctricas para comprometer secretos.

USB y periféricos

Dispositivos maliciosos, BadUSB o medios removibles pueden inyectar código o extraer información.

Puertos de depuración

JTAG u otras interfaces habilitadas exponen funciones privilegiadas del hardware.



Las interfaces físicas son útiles para mantenimiento, pero también amplían la superficie de ataque.

Fuentes: Mell, P., & Bojanova, I. (2024). NIST IR 8517; CISA. (2024). How to Protect the Data that is Stored on Your Devices.

1.2.1. Amenazas a nivel de Hardware

Firmware, UEFI y arranque seguro

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Amenazas críticas

Los ataques al firmware se ejecutan antes del sistema operativo y pueden lograr persistencia, evasión de controles y control total del equipo.

Bootkits y rootkits UEFI

Modifican el proceso de arranque para cargar código malicioso.

Actualizaciones y rollback

Firmware no firmado o versiones antiguas facilitan la explotación.

Option ROMs y claves

Componentes de prearranque y claves desactualizadas debilitan la cadena de confianza.



En 2026, la actualización de certificados de Secure Boot es un punto de atención operativo.

La defensa depende de firmware firmado, Secure Boot, medición del arranque y recuperación segura.

Fuentes: Regenscheid, A. (2018). NIST SP 800-193; Microsoft Learn. (2026). Secure boot.

1.2.1. Amenazas a nivel de Hardware

Impactos y buenas prácticas de mitigación

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Impactos

Confidencialidad
Robo de datos, credenciales y claves.

Integridad
Alteración del firmware, del arranque o de la configuración.

Disponibilidad
Fallas graves, interrupción del servicio o inutilización del equipo.

Costos
Reposición, respuesta al incidente y daño reputacional.

Mitigación

- Mantener firmware y software actualizados.
- Habilitar Secure Boot, TPM y mecanismos de raíz de confianza.
- Restringir puertos, medios removibles y acceso físico.
- Usar cifrado, copias de seguridad y monitoreo de integridad.
- Evaluar proveedores y exigir arranque y recuperación seguros.

La seguridad del hardware requiere prevención, detección, recuperación y control de la cadena de suministro.

Fuentes: Regenscheid, A. (2018). NIST SP 800-193; Mell, P., & Bojanova, I. (2024). NIST IR 8517; ENISA. (2025). ENISA Threat Landscape 2025.

1.2.2. Amenazas a nivel de Código [1/6]

Visión general

¿Qué son?

Son debilidades en el código fuente, librerías o manejo de datos que permiten acceso no autorizado, alteración, ejecución indebida o interrupción del sistema.

Riesgos frecuentes

- Inyección y XSS
- Control de acceso y autenticación
- Fallas criptográficas
- Errores de memoria
- Integridad y cadena de suministro



Contexto actual

OWASP Top 10:2025 prioriza control de acceso, cadena de suministro, fallas criptográficas, inyección e integridad del software/datos.

Tendencia 2026

El DBIR 2026 reporta que el **31%** de las brechas analizadas inició con explotación de vulnerabilidades.

Impacto

Pérdida de confidencialidad, fraude, indisponibilidad, compromiso de cuentas y afectación reputacional.

La seguridad a nivel de código exige prevención desde el desarrollo, no solo corrección posterior.

Fuentes: OWASP Foundation. (2025). OWASP Top 10:2025; Verizon. (2026). Data Breach Investigations Report; National Institute of Standards and Technology. (2022). SP 800-218.

1.2.2. Amenazas a nivel de Código [2/6]

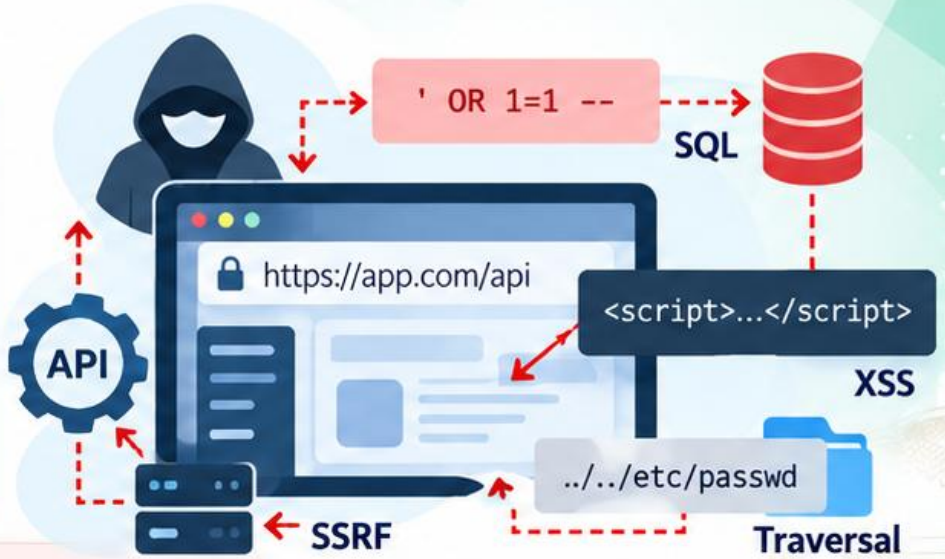
Inyección y validación insuficiente

Idea clave

Cuando la aplicación interpreta datos no confiables como instrucciones, el atacante puede alterar consultas, ejecutar acciones no previstas o exponer información.

Inyección

- SQL/NoSQL/LDAP/OS command
- Manipula consultas o comandos
- Puede leer, alterar o borrar datos.



XSS y plantillas

- Inyecta scripts en el navegador
- Roba sesiones o modifica contenido
- Afecta formularios, comentarios y paneles.

SSRF y traversal

- Fuerza al servidor a consultar recursos internos
- Expone metadata o servicios internos
- Traversal accede a archivos fuera de la ruta prevista.

Controles esenciales

- Validación en servidor
- Consultas parametrizadas
- Escapado/contextualización de salida
- Principio de mínimo privilegio.

Contexto actual

OWASP Top 10:2025 mantiene la inyección entre los riesgos críticos y refuerza la necesidad de controles en APIs y aplicaciones web.

La entrada del usuario nunca debe tratarse como confianza implícita: **validar, parametrizar y codificar la salida es esencial.**

Fuentes: OWASP Foundation. (2025). OWASP Top 10:2025; National Institute of Standards and Technology. (2022). SP 800-218.

1.2.2. Amenazas a nivel de Código [3/6]

Acceso, autenticación y fallas criptográficas

Pérdida de control de acceso

Fallos de autorización permiten leer, modificar o eliminar recursos ajenos. Incluye IDOR/BOLA, escalación de privilegios y validaciones ausentes por endpoint.

Fallas de autenticación

- Contraseñas débiles o reutilizadas
- Sesiones y tokens mal gestionados
- MFA ausente o mal implementado.



Fallas criptográficas

- Datos sensibles sin cifrado
- Algoritmos o configuraciones débiles
- Gestión insegura de llaves.

Secretos expuestos

- Credenciales hardcoded
- Claves en repositorios o archivos .env mal protegidos
- Tokens con exceso de permisos.

Consecuencias

- Toma de cuentas
- Fraude
- Exfiltración de datos
- Incumplimiento normativo.

OWASP Top 10:2025 ubica como riesgos principales el control de acceso, las fallas criptográficas y las fallas de autenticación.

La identidad y los secretos deben protegerse desde el código: autorización por defecto, cifrado correcto y gestión segura de credenciales.

Fuentes: OWASP Foundation. (2025). OWASP Top 10:2025; Cybersecurity and Infrastructure Security Agency & Federal Bureau of Investigation. (2025). Product Security Bad Practices; National Institute of Standards and Technology. (2022). SP 800-218.

1.2.2. Amenazas a nivel de Código [4/6]

Errores de memoria y programación insegura

Desbordamiento de búfer

Escrituras fuera de límites sobrescriben memoria y pueden provocar ejecución arbitraria o caída del sistema.

Use-after-free / double free

El programa reutiliza memoria liberada o la libera incorrectamente, generando corrupción y ejecución impredecible.



Integer overflow y lógica insegura

Cálculos erróneos de tamaño, índices o límites pueden abrir puertas a lecturas/escrituras indebidas.

Condiciones excepcionales

El manejo inadecuado de errores, excepciones y estados inesperados puede desactivar controles o revelar datos.

Contexto moderno

CISA y sus socios advierten que alrededor de dos tercios de las vulnerabilidades reportadas en lenguajes no seguros en memoria se relacionan con problemas de memoria.

Migrar componentes críticos hacia lenguajes memory-safe reduce el riesgo estructural.

La seguridad del código no depende solo de corregir bugs: también exige **decisiones de lenguaje, revisión y pruebas de robustez.**

Fuentes: Cybersecurity and Infrastructure Security Agency et al. (2023). The Case for Memory Safe Roadmaps; Cybersecurity and Infrastructure Security Agency et al. (2024). Exploring Memory Safety in Critical Open Source Projects; OWASP Foundation. (2025). OWASP Top 10:2025.

1.2.2. Amenazas a nivel de Código [5/6]

Integridad del software y cadena de suministro

Dependencias vulnerables

Bibliotecas desactualizadas o con CVE conocidas heredan riesgo al producto final.

Paquetes maliciosos

Typosquatting, dependency confusion y paquetes comprometidos pueden introducir código dañino durante la instalación.



Integridad del build

Artefactos no firmados, pipelines CI/CD comprometidos o scripts inseguros facilitan la alteración del software.

Transitividades y SBOM

Las dependencias indirectas amplían la exposición; sin inventario claro, la remediación se retrasa.

Tendencia moderna

OWASP Top 10:2025 incorpora Fallas en la Cadena de Suministro de Software, y el DBIR 2026 reporta crecimiento de incidentes asociados a terceros y dependencias.

Hoy la seguridad del código también depende de **paquetes, repositorios** y **automatizaciones de entrega.**

Asegurar el código exige asegurar también sus **dependencias, su proceso de construcción** y **la trazabilidad del software.**

Fuentes: OWASP Foundation. (2025). OWASP Top 10:2025; Verizon. (2026). Data Breach Investigations Report; Cybersecurity and Infrastructure Security Agency & Federal Bureau of Investigation, (2025). Product Security Bad Practices.

1.2.2. Amenazas a nivel de Código [6/6]

Mitigación y buenas prácticas

Diseño seguro

- Modelado de amenazas.
- Requisitos de seguridad desde el inicio.
- Principios de secure-by-design.

Codificación segura

- Validación de entrada.
- Consultas parametrizadas.
- Manejo seguro de errores.
- Revisión de código.

Pruebas automáticas

- SAST (análisis estático).
- DAST (análisis dinámico).
- SCA (dependencias).
- Fuzzing.
- Pruebas de seguridad en APIs.



Secretos y acceso

- Uso de vaults de secretos.
- MFA para accesos críticos.
- Rotación de credenciales.
- Mínimo privilegio.
- Control de sesiones y tokens.

Cadena de suministro

- Fijar versiones de dependencias.
- Verificar firmas e integridad.
- Usar SBOM (lista de materiales).
- Monitorear dependencias vulnerables.

Respuesta y mejora

- Registro y monitoreo (logging).
- Alertas y detección temprana.
- Parcheo ágil.
- Programa de divulgación de vulnerabilidades (VDP).
- Lecciones aprendidas.

El **SSDF de NIST** orienta prácticas para reducir vulnerabilidades desde el ciclo de vida de desarrollo.

La defensa efectiva a nivel de código combina **prevención, automatización** y **mejora continua** en todo el SDLC.

Fuentes: National Institute of Standards and Technology. (2022). SP 800-218; OWASP Foundation. (2025). OWASP Top 10:2025; Cybersecurity and Infrastructure Security Agency. (2025). Secure by Design / Product Security Bad Practices.

1.2.2. Amenazas a nivel de Código

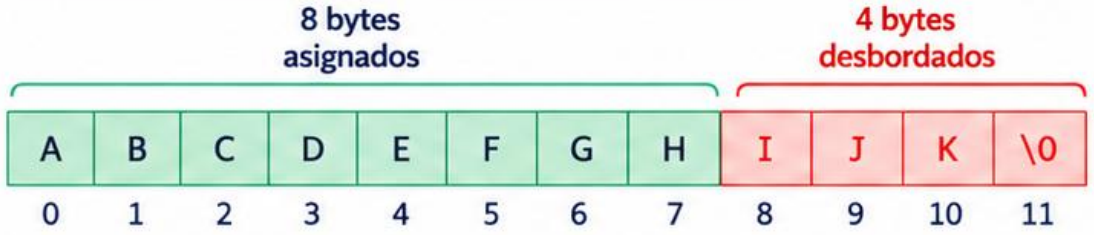
ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Ejemplos prácticos de desbordamiento de búfer

1 Entrada sin límite (scanf)

```
// Código vulnerable (C)
char nombre[8];
scanf("%s", nombre);
```

Entrada: "ABCDEFGHIJK" (11 caracteres)
Necesita **12 bytes** con \0; solo hay **8 bytes**.



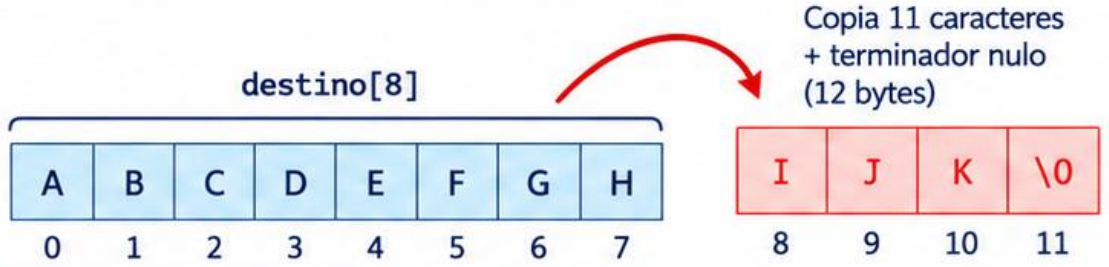
Código corregido (C)

```
char nombre[8];
scanf("%7s", nombre);
```

Lee como máximo 7 caracteres y deja espacio para el terminador nulo (\0).

2 Copia sin verificar tamaño (strcpy)

```
// Código vulnerable (C)
char destino[8];
strcpy(destino, "ABCDEFGHIJK");
```



Código corregido (C)

```
char destino[8];
snprintf(destino, sizeof(destino), "%s", "ABCDEFGHIJK");
```

Copia de forma segura, truncando el contenido para no exceder la capacidad del búfer.

Escribir fuera de los límites del búfer puede corromper datos, bloquear el programa o facilitar la ejecución de código no autorizado.

Fuente: MITRE. (s. f.). CWE-120: Buffer Copy without Checking Size of Input. https://cwe.mitre.org/data/definitions/120

1.2.3. Lenguajes de script incrustados [1/4]

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

¿Qué son?

Código interpretado integrado en páginas, plantillas o aplicaciones para ejecutar funciones dinámicas.

Navegador

JavaScript incrustado en HTML, SPA y componentes web.

Servidor

Plantillas dinámicas como Jinja2 y Twig en aplicaciones web.

Aplicaciones y automatización

Intérpretes como Lua o Python en scripts y tareas automatizadas.

Entrada no confiable

Datos de usuarios, archivos, solicituides, etc.

Interpretación como código

El script se ejecuta en el navegador, servidor o aplicación.

Riesgo de ejecución indebida

Ejecución de código malicioso, acceso no autorizado, robo de datos, entre otros.

Riesgos comunes: XSS | Inyección de plantillas (SSTI) | eval() inseguro

El riesgo surge al confundir datos con instrucciones ejecutables.

Fuentes: OWASP Foundation (2025); OWASP Cheat Sheet Series (s. f.).

1.2.3. Lenguajes de script incrustados [2/4]

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Amenazas modernas y defensas (2026)

Amenaza	Control recomendado
DOM XSS: innerHTML y HTML no confiable.	textContent, sanitización contextual y Trusted Types.
eval() y new Function(): ejecución dinámica.	Eliminar eval, validación y CSP estricta.
Scripts de CDN o terceros alterados.	SRI (integrity), versiones fijadas e inventario.
Plantillas dinámicas con instrucciones inyectadas.	Separar datos y plantillas; autoescape.

Novedad 2026: Trusted Types alcanza compatibilidad Baseline en navegadores modernos (MDN Web Docs).

Defensa en profundidad = codificación segura + CSP + gestión de dependencias.

Fuentes: OWASP Foundation (2025); OWASP Cheat Sheet Series (s. f.); MDN Web Docs (s. f.).

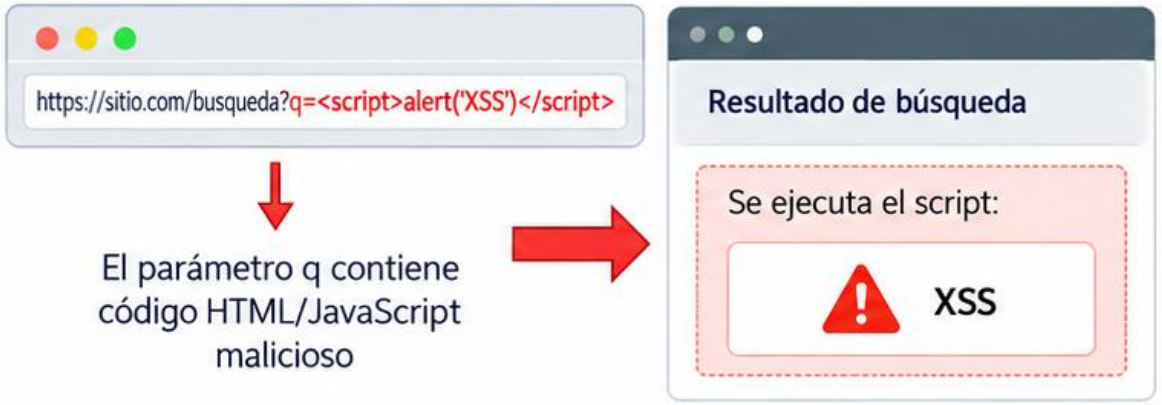
1.2.3. Lenguajes de script incrustados [3/4]

UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Ejemplo 1 · DOM XSS en JavaScript

Código vulnerable

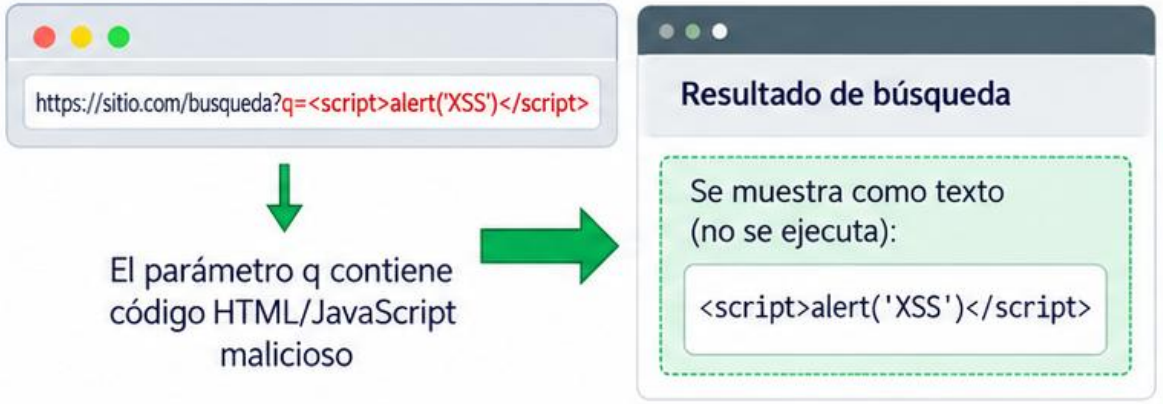
```
1  const q = new URLSearchParams(location.search).get('q');
2  document.querySelector('#salida').innerHTML = q;
```



Riesgo: XSS, modificación de la página y posible secuestro de sesión.

Código corregido

```
1  const q = new URLSearchParams(location.search).get('q');
2  document.querySelector('#salida').textContent = q;
```



No insertar entradas no confiables en innerHTML; usar **textContent**.

Fuente: OWASP Cheat Sheet Series. (s. f.). *DOM Based XSS Prevention*.

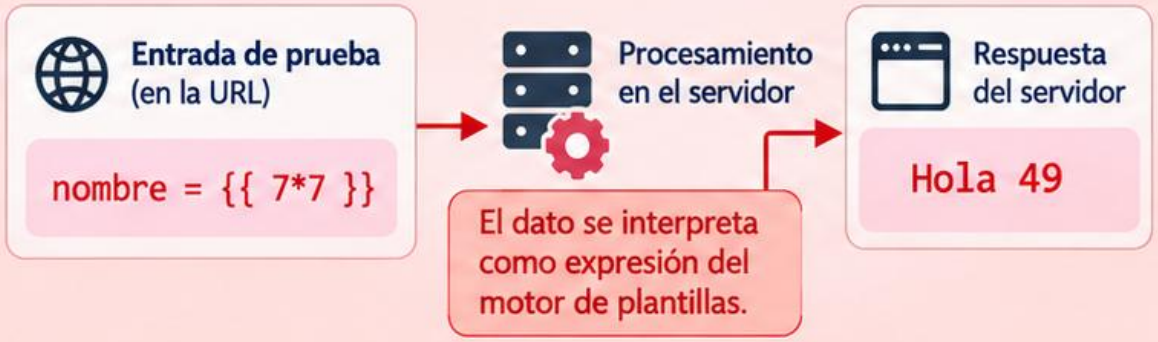
1.2.3. Lenguajes de script incrustados [4/4]

UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Ejemplo 2 · Inyección de plantillas (SSTI)

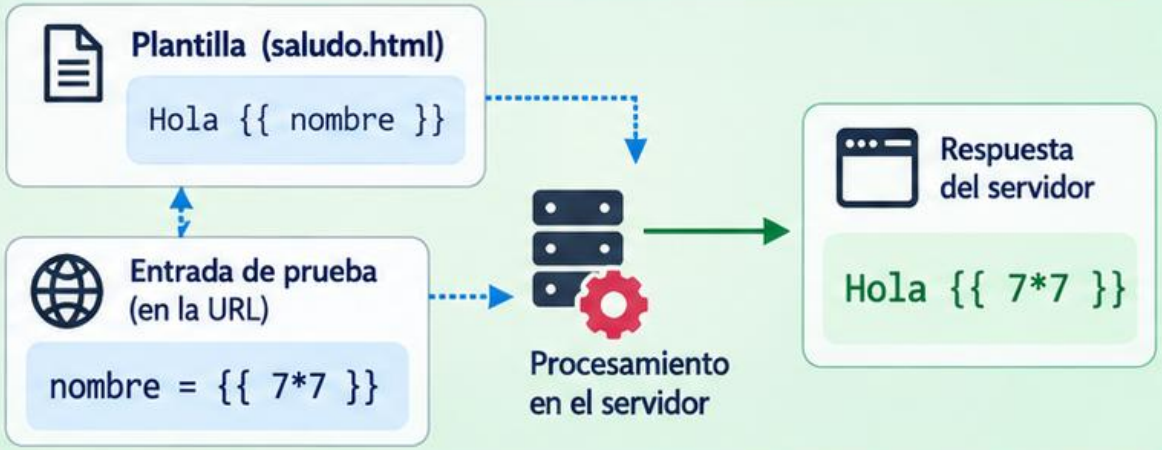
Código vulnerable (Flask/Jinja2)

```
nombre = request.args.get("nombre", "")
html = "Hola " + nombre
return render_template_string(html)
```



Código corregido (Flask/Jinja2)

```
nombre = request.args.get("nombre", "")
return render_template("saludo.html", nombre=nombre)
```



Conclusión: Separar datos y plantilla evita la evaluación de expresiones no confiables.

El **autoescape HTML no sustituye** la separación entre código y datos.

Fuente: PortSwigger (s. f.), Server-side template injection; Jinja Documentation (s. f.).

1.2.4. Amenazas a nivel de diseño [1/5]

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Visión general: fallos que nacen antes de programar

Un diseño inseguro omite controles esenciales desde los requisitos, los flujos y la lógica de negocio.

Una implementación impecable no corrige un control que nunca fue previsto.



Lógica de negocio vulnerable

Ej.: una tienda permite aplicar ilimitadamente un cupón de descuento.

Control: invariantes, límites y estados de negocio.

Recuperación de cuenta insegura

Ej.: restablecer claves respondiendo una pregunta personal.

Control: recuperación con prueba de identidad robusta.

Confianza indebida en el cliente

Ej.: el navegador decide el precio final de la compra.

Control: decisiones sensibles en el servidor.

Fuentes: OWASP Foundation (2025), A06: Diseño inseguro; NIST (2022), SP 800-218.

1.2.4. Amenazas a nivel de diseño [2/5]

Modelado de amenazas y casos de abuso

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

1. Modelar flujos
Comprender procesos, activos y límites de confianza.

2. Identificar amenazas
Analizar posibles escenarios de ataque.

3. Priorizar riesgos
Evaluar impacto y probabilidad.

4. Verificar controles
Definir e implementar medidas de mitigación.

Suplantación (STRIDE)

Ej.: token de sesión reutilizado para hacerse pasar por otro usuario.

Control: vincular sesión, MFA y revocación.

Manipulación (STRIDE)

Ej.: modificar cantidad o precio durante el pago.

Control: validar invariantes en backend.

Elevación de privilegios (STRIDEE)

Ej.: un usuario invoca una acción exclusiva del administrador.

Control: autorización por recurso y función.

Caso de abuso: ¿qué pasa si el atacante salta pasos, repite solicitudes o cambia el orden del flujo?

Fuentes: OWASP Foundation (s. f.), Threat Modeling Cheat Sheet; OWASP Foundation (2025), A06 Diseño inseguro.

1.2.4. Amenazas a nivel de diseño [3/5]

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Secure by Design: patrones, requisitos y normas

Ausencia de requisitos de seguridad

Ej.: historia de usuario de compras sin límite de intentos.

Control: criterios de aceptación de seguridad y casos de abuso.

Permisos excesivos por defecto

Ej.: cuenta nueva con rol administrador.

Control: denegar por defecto y mínimo privilegio.

Control de acceso solo en el cliente

Ej.: ocultar botón de eliminar, pero la API lo permite.

Control: mediación completa y autorización en backend.

Patrones de diseño seguro: mínimo privilegio · denegar por defecto · mediación completa · separación de responsabilidades.

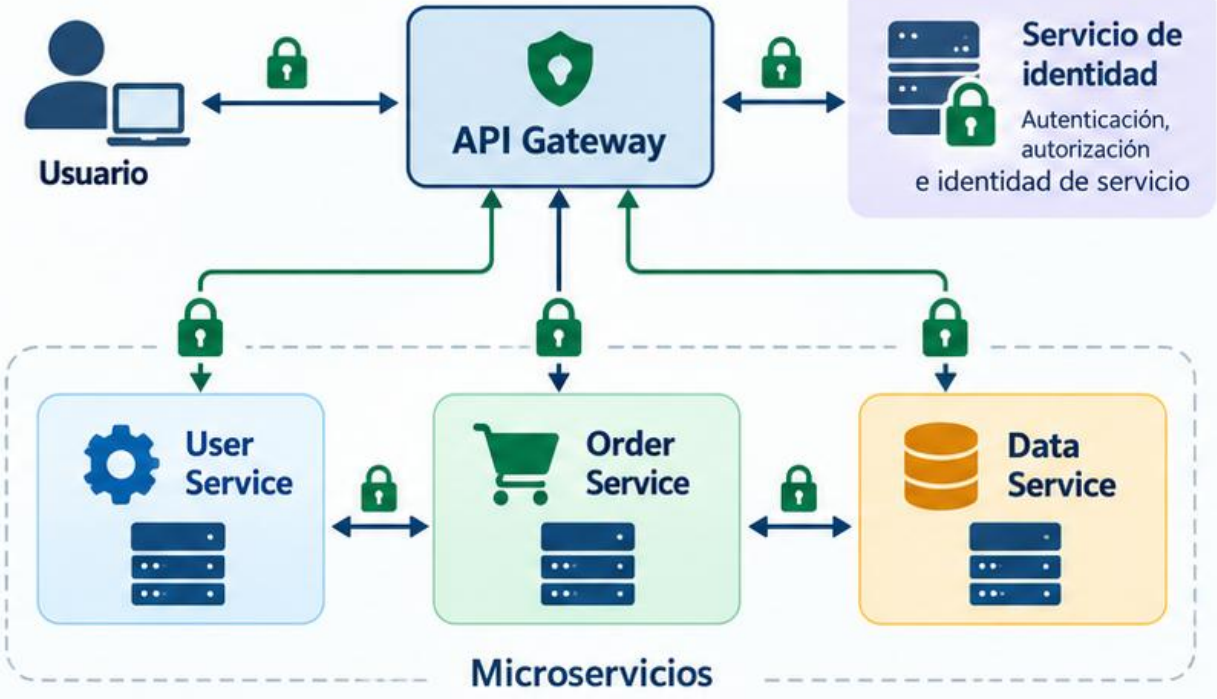
ISO/IEC 27034-1 integra seguridad de aplicaciones; **ISO/IEC 27001:2022** gestiona riesgos; NIST SSDF incorpora prácticas al ciclo de desarrollo.

Nota: los patrones de diseño de propósito general (p. ej., *Factory Method* o *Singleton*) *no son patrones de seguridad por sí mismos*, aunque pueden contribuir cuando se aplican con requisitos de seguridad.

Fuentes: OWASP Foundation (2025), A06: Diseño inseguro; ISO/IEC (2011), 27034-1; ISO/IEC (2022), 27001; NIST (2022), SP 800-218.

1.2.4. Amenazas a nivel de diseño [4/5]

Zero Trust y aislamiento de aplicaciones



Nunca confiar por ubicación; verificar cada acceso.

1. Confianza implícita en red interna

Ejemplo: Order Service acepta llamadas sin autenticar porque provienen de la VLAN interna.

Control: identidad de servicio + autenticación mutua mTLS + autorización.

2. Acceso entre organizaciones (tenants)

Ejemplo: /facturas/42 expone datos de otra empresa.

Control: autorización por objeto y tenant en cada consulta.

3. Privilegios entre microservicios

Ejemplo: un servicio de catálogo puede cancelar órdenes.

Control: mínimo privilegio y políticas por función/recurso.

Zero Trust = verificar identidad, contexto y permisos en cada solicitud. No basta un API Gateway.

Rose et al. (2020), NIST SP 800-207; Chandramouli y Butcher (2023), NIST SP 800-207A.

1.2.4. Amenazas a nivel de diseño [5/5]

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Diseño seguro ante IA generativa y agentes autónomos (2026)

Secuestro del objetivo (ASI01)

EJEMPLO

Un documento recuperado incluye instrucciones para que el agente envíe información fuera del sistema.

CONTROL

Tratar documentos como datos no confiables, validar acciones y destinos.

Abuso de herramientas (ASI02)

EJEMPLO

Un asistente que solo debía consultar pedidos ejecuta una operación de eliminación.

CONTROL

Lista de herramientas permitidas y confirmación humana para acciones críticas.

Abuso de identidad y privilegios (ASI03)

EJEMPLO

El agente usa un token con acceso a todas las organizaciones.

CONTROL

Credenciales efímeras, mínimos privilegios y autorización por tenant.



Diseñar agentes seguros = límites de confianza + autorizaciones + supervisión + trazabilidad.

ISO/IEC 42001:2023 — gestión de IA
Marco de gestión para organizaciones que desarrollan o utilizan sistemas de IA.

NIST AI RMF — gestión de riesgos
Marco para identificar, evaluar, gestionar y monitorear riesgos de IA.

Fuentes: OWASP GenAI Security Project (2025), Top 10 for Agentic Applications 2026; ISO/IEC (2023), 42001; NIST (2024), AI 600-1.